

ปัญญาประดิษฐ์ยุคปัจจุบัน

สัปดาห์ที่ 13: Harness, AI Skills และ Plugins

ผศ. ดร. อธิพล ฟองแก้ว
ittipon@g.sut.ac.th

วันนี้

สัปดาห์ที่แล้วเราเขียนลูปเอเจนต์เอง สัปดาห์นี้เราจะดูว่าเครื่องมือระดับใช้งานจริง สร้างลูปนั้นอย่างไร และเราจะ**ขยายความสามารถ**มันได้อย่างไรโดยไม่ต้องเขียนเอเจนต์ใหม่

ชั่วโมงที่ 1

กายวิภาคของ harness

- harness คืออะไร
- พรอมป์ระบบและเครื่องมือ
- ชั้นสิทธิ์
- ตัวจัดการบริบทและ hooks

ชั่วโมงที่ 2

Agent Skills

- ปัญหาที่ skills แก้
- การเปิดเผยแบบขั้นบันได
- โครงสร้างของ skill
- เขียน description ให้ดี

ชั่วโมงที่ 3

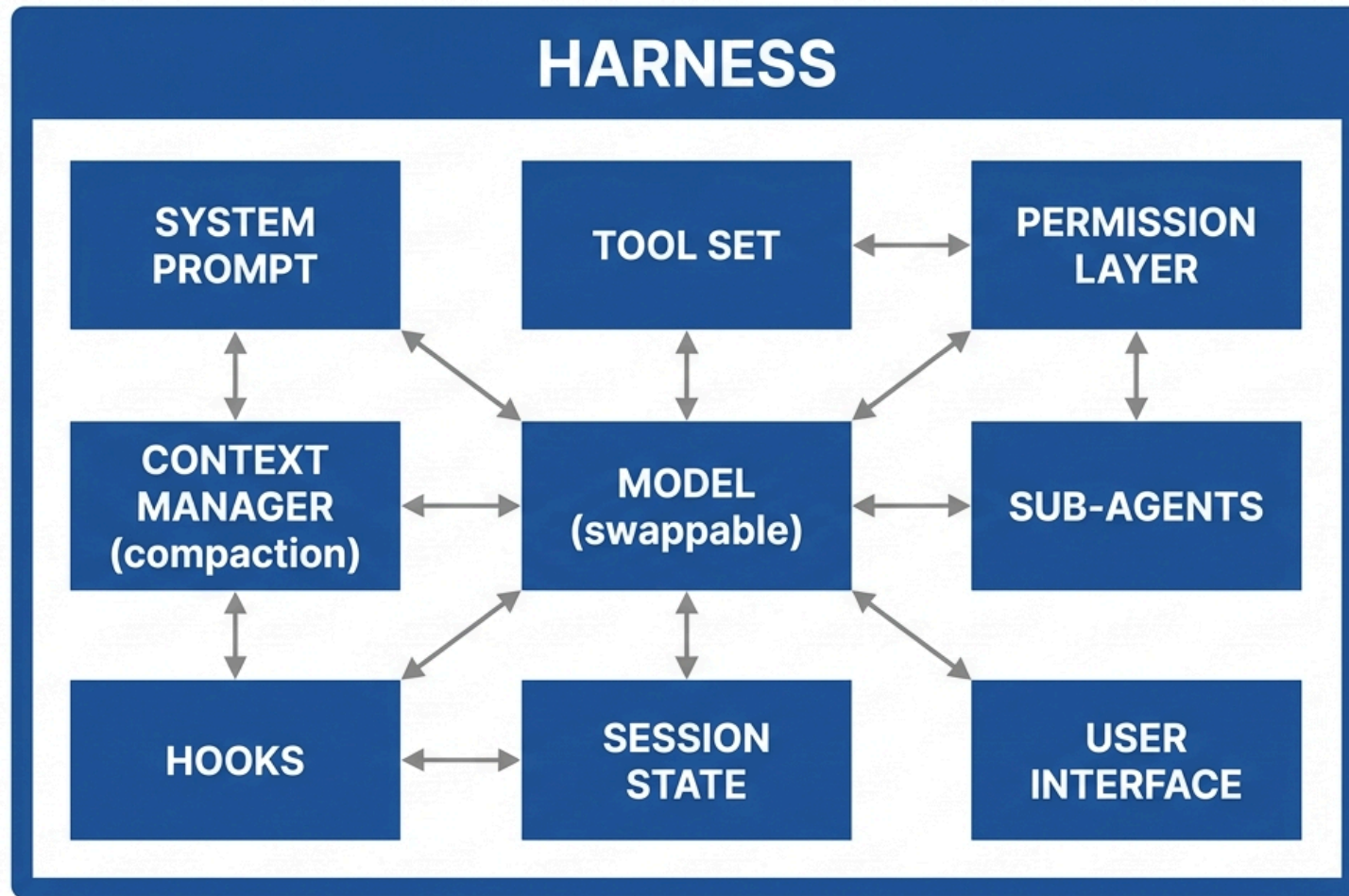
Plugins และการเลือก

- รวมเป็นชุดแจกจ่าย
- คำสั่งลัดและ hooks
- เลือกกลไกให้ถูก
- ลงมือทำในแล็บ

ทำไมเรื่องนี้สำคัญกับนักวิทยาการคอมพิวเตอร์: ในอีก 2 ถึง 3 ปี งานส่วนใหญ่ของคุณจะไม่ใช่การเขียนเอเจนต์ตั้งแต่ศูนย์ แต่คือการ**ปรับแต่ง harness** ที่มีอยู่ ให้ทำงานเฉพาะทางขององค์กรคุณได้

ชั่วโมงที่ 1

กายวิภาคของ harness



the same model in a different harness behaves very differently

นิยาม

Harness คือโปรแกรมที่ห่อหุ้มโมเดล และแปลงมันจาก "ตัวทำนายโทเคน" ให้เป็น "ผู้ช่วยที่ทำงานได้จริง"

ข้อสังเกตที่สำคัญที่สุดของสัปดาห์นี้: ความสามารถของผู้ช่วย AI ที่คุณใช้ ไม่ได้มาจากโมเดลเพียงอย่างเดียว โมเดลตัวเดียวกัน ใน harness ที่ต่างกัน ให้ผลลัพธ์ต่างกันอย่างมาก

หลักฐานเชิงประจักษ์: โมเดลเดียวกันที่ทำคะแนน benchmark การเขียนโค้ดได้ต่างกัน 20 ถึง 30 จุด เมื่อรันภายใต้ harness ที่ต่างกัน ความต่างมาจากชุดเครื่องมือ การจัดการบริบท และพารามิเตอร์ระบบ ล้วน ๆ

นัยเชิงอาชีพ: ถ้าความสามารถมาจาก harness พอ ๆ กับตัวโมเดล แปลว่าคนที่เข้าใจและปรับแต่ง harness เป็น มีคุณค่าไม่ต่างจากคนที่ฝึกโมเดลเป็น และเข้าถึงได้ง่ายกว่ามาก เพราะไม่ต้องมี GPU cluster

ตัวอย่าง harness ที่ควรรู้จัก

Harness	รูปแบบ	โมเดลที่ใช้ได้	โอเพนซอร์ส
Claude Code	เทอร์มินัล, IDE	Claude	ไม่ (แต่ขยายได้เต็มที่)
Gemini CLI	เทอร์มินัล	Gemini	ใช่
Qwen Code	เทอร์มินัล	Qwen และอื่น ๆ	ใช่
OpenAI Codex CLI	เทอร์มินัล	GPT	ใช่
Cline / Roo Code	ส่วนขยาย VS Code	ต่อ API ใดก็ได้	ใช่
Aider	เทอร์มินัล	ต่อ API ใดก็ได้	ใช่
OpenHands	เว็บ, คอนเทนเนอร์	ต่อ API ใดก็ได้	ใช่
Cursor / Windsurf	ตัวแก้ไขโค้ด	หลายโมเดล	ไม่

กายวิภาค: 1. พรอมป์ตรระบบและเครื่องมือ

พรอมป์ตรระบบ ของ harness ระดับใช้งานจริงมักยาวหลายพัน โทเคน ประกอบด้วย

- บทบาทและน้ำเสียง
- คำอธิบายสภาพแวดล้อม (OS, ไดรกทอรี, สถานะ git)
- กติกาความปลอดภัยและขอบเขตที่ห้ามข้าม
- แนวทางการใช้เครื่องมือแต่ละตัว
- ข้อกำหนดรูปแบบผลลัพธ์
- วิธีจัดการเมื่อไม่แน่ใจ

ชุดเครื่องมือหลัก ที่แทบทุก coding harness มีเหมือนกัน

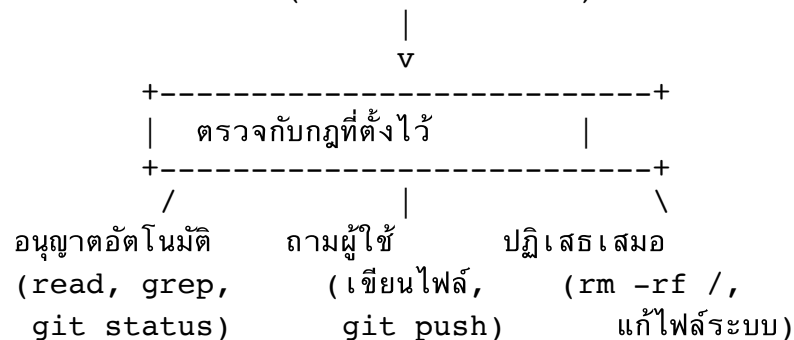
- อ่านไฟล์, เขียนไฟล์, แก้ไขบางส่วน
- ค้นด้วยชื่อไฟล์ (glob) และเนื้อหา (grep)
- รันคำสั่งเชลล์
- ค้นเว็บ, ดึงหน้าเว็บ
- จัดการรายการงาน

สังเกต: read, edit, grep, bash สี่ตัวนี้ครอบคลุมงานเขียนโปรแกรมได้เกือบทั้งหมด นี่คือการออกแบบเครื่องมือจาก
สัปดาห์ที่ 11 ที่ใช้จริง: เครื่องมือน้อย ทรงพลัง และประกอบกันได้ ชนะเครื่องมือเยอะที่เฉพาะเจาะจงเกินไป

กายวิภาค: 2. ชั้นสิทธิ์

จุดที่แยก harness ระดับของเล่นออกจากระดับใช้งานจริง

โมเดลขอเรียก: `bash("rm -rf build/")`



โหมดที่มักมีให้เลือก

- ถามทุกครั้งที่จะเปลี่ยนแปลงอะไร (ค่าเริ่มต้น)
- อนุญาตแก้ไขไฟล์อัตโนมัติ แต่ถามก่อนรันคำสั่ง
- อนุญาตทุกอย่าง (ใช้เฉพาะใน sandbox หรือ CI เท่านั้น)

อย่ารันโหมดอนุญาตทุกอย่างบนเครื่องที่มีข้อมูลจริง ให้ใช้คอนเทนเนอร์ หรือสำเนาที่ทิ้งได้ นี่คือหลัก sandbox จาก สัปดาห์ที่แล้ว ในรูปแบบที่ใช้งานจริง

แบบฝึกหัด 1.1: ออกแบบนโยบายสิทธิ์

คุณตั้งค่า harness ให้ทีมนักศึกษาปริญญาโทใช้กับโค้ดงานวิจัยในเครื่องของแล็บ

จัดแต่ละคำสั่งลงใน 3 กลุ่ม: **อนุญาตอัตโนมัติ** / **ถามก่อน** / **ปฏิเสธเสมอ**

#	คำสั่ง
ก	<code>pytest tests/</code>
ข	<code>git commit -m "..."</code>
ค	<code>git push origin main</code>
ง	<code>rm -rf results/</code>
จ	<code>pip install <package></code>
ฉ	<code>sbatch job.sh</code> (ส่งงานเข้าคิว HPC ที่คิดเงินตามชั่วโมง)
ช	<code>curl -X POST https://api.example.com -d @data.json</code>

เฉลย 1.1

#	คำสั่ง	กลุ่ม	เหตุผล
ก	<code>pytest tests/</code>	อัตโนมัติ	อ่านอย่างเดียว ย้อนกลับได้ และเป็นวิธีตรวจสอบความจริงภายนอก ที่เราอยากให้ทำบ่อย ๆ
ข	<code>git commit</code>	อัตโนมัติ	ย้อนกลับได้ง่ายด้วย <code>git reset</code> และยังไม่ออกจากเครื่อง
ค	<code>git push origin main</code>	ถามก่อน	ออกสู่ภายนอก ย้อนกลับยาก กระทบคนอื่นในทีม
ง	<code>rm -rf results/</code>	ถามก่อน	ลบข้อมูลที่อาจใช้เวลาคำนวณเป็นวัน ย้อนกลับไม่ได้
จ	<code>pip install</code>	ถามก่อน	เปลี่ยนสภาพแวดล้อม อาจพังงานคนอื่น และเป็นช่องทาง supply chain attack
ฉ	<code>sbatch job.sh</code>	ถามก่อน	ใช้เงินจริง ตามชั่วโมง GPU
ช	<code>curl -X POST ... -d @data.json</code>	ปฏิเสธเสมอ	ส่งข้อมูลออกนอกองค์กร คือชาติที่ 3 ของกฎหมายการ

ข้อ ช คือข้อที่คนพลาดบ่อยที่สุด หลายคนจัดเป็น "ถามก่อน" แต่ถ้าเเจนต์อ่านข้อมูลวิจัยได้ และเจอ prompt injection จากไฟล์ที่ดาวน์โหลดมา คำขออนุมัติจะดูสมเหตุสมผลจนผู้ใช้กดผ่าน การปฏิเสธที่ระดับระบบแข็งแกร่งกว่าการฟัง วิจารณ์ญาณของมนุษย์ที่กำลังรีบ

กายวิภาค: 3. ตัวจัดการบริบท

งานที่ยากที่สุดของ harness คือจัดการหน้าต่างบริบทที่มีจำกัด

กลไก	ทำอะไร
การบีบอัด (compaction)	เมื่อบริบทใกล้เต็ม สรุปการสนทนาเก่าเป็นบทสรุป แล้วทำงานต่อ
ไฟล์คำสั่งประจำโปรเจกต์	CLAUDE.md, AGENTS.md, GEMINI.md โหลดอัตโนมัติทุกเซสชัน
การอ่านแบบเลือกส่วน	อ่านไฟล์เฉพาะช่วงบรรทัด แทนที่จะโหลดทั้งไฟล์
การตัดผลลัพธ์เครื่องมือ	ตัดผลลัพธ์ยาว ๆ และเก็บส่วนเต็มไว้ในไฟล์แทน
การถ่ายลงไฟล์	เขียนแผนหรือความคืบหน้าลงไฟล์ แล้วอ่านกลับเมื่อต้องใช้

นี่คือกลยุทธ์ 5 แบบจากสัปดาห์ที่ 9 ที่ถูกนำมาใช้จริง AGENTS.md เป็นข้อตกลงกลางที่เครื่องมือหลายค่ายรองรับ ใส่กติกาของโปรเจกต์ไว้ในนั้น เช่น วิธีรันเทส รูปแบบโค้ด ข้อห้าม แล้วผู้ช่วย AI ทุกตัวที่ทีมคุณใช้จะเห็นเหมือนกัน

กายวิภาค: 4. เอเจนต์ย่อยและ hooks

เอเจนต์ย่อย (Subagents)

งานสำรวจที่ต้องอ่านข้อมูลมาก ถูกส่งไปทำในบริบทแยก แล้วส่งกลับมาแค่ข้อสรุป

หลัก: "หาว่า auth ทำงานอย่างไร"

--> ย่อย: อ่าน 30 ไฟล์

--> กลับมา: สรุป 15 บรรทัด

บริบทของเอเจนต์หลักเหลือพื้นที่ทำงานต่อ **นี่คือรูปแบบผู้ประสานงานกับผู้ปฏิบัติจากสัปดาห์ที่แล้ว**

Hooks

โค้ดของเราที่ถูกเรียกอัตโนมัติ ณ จุดที่กำหนด

จุด	ใช้ทำอะไร
ก่อนใช้เครื่องมือ	บล็อกคำสั่งอันตราย
หลังใช้เครื่องมือ	รัน formatter, linter
เริ่มเซสชัน	ใส่บริบทของโปรเจกต์
ก่อนปิดบริบท	บันทึกสถานะไว้

hooks เป็น **โปรแกรมจริง** จึงบังคับใช้กฎได้แน่นอน

หลักการที่ต้องจำ: อะไรที่ ต้อง เกิดขึ้นทุกครั้ง ให้ทำด้วย hook ไม่ใช่ด้วยการเขียนใส่พรอมต์
พรอมต์คือการขอร้อง hook คือการบังคับ

สรุปชั่วโมงที่ 1

- **Harness** คือโปรแกรมรอบโมเดล และเป็นตัวกำหนดความสามารถจริงพอ ๆ กับตัวโมเดลเอง
- องค์ประกอบหลัก: พรอมป์ระบบ, เครื่องมือ, ชั้นสิทธิ์, ตัวจัดการบริบท, เอเจนต์ย่อย, hooks
- **เครื่องมือย่อยที่ทรงพลังและประกอบกันได้** ชนะเครื่องมือเยอะที่เฉพาะเจาะจงเกินไป
- **ชั้นสิทธิ์คือสิ่งที่แยก harness ระดับใช้งานจริงออกจากของเล่น**
- **การปฏิเสธที่ระดับระบบ แข็งแรงกว่าการฟังวิจารณ์จากผู้ใชที่กำลังรีบ**
- **พรอมป์คือการขอร้อง hook คือการบังคับ**

พัก 10 นาที

ชั่วโมงที่ 2

Agent Skills

ปัญหา: จะสอนความรู้เฉพาะทางให้เอเจนต์อย่างไร

สมมติเล็บของคุณมีขั้นตอนเฉพาะ เช่น วิธีตั้งค่าไฟล์อินพุตของ โปรแกรมคำนวณ

ทางเลือกที่ไม่ดี

- ใส่ทุกอย่างในพรมป้ระบบ
กินบริบทตลอดเวลาแม้ไม่ได้ใช้
- fine-tune โมเดล
แพง ช้า และแก้ไขยาก
- อธิบายใหม่ทุกครั้ง
ไม่สม่ำเสมอและเสียเวลา

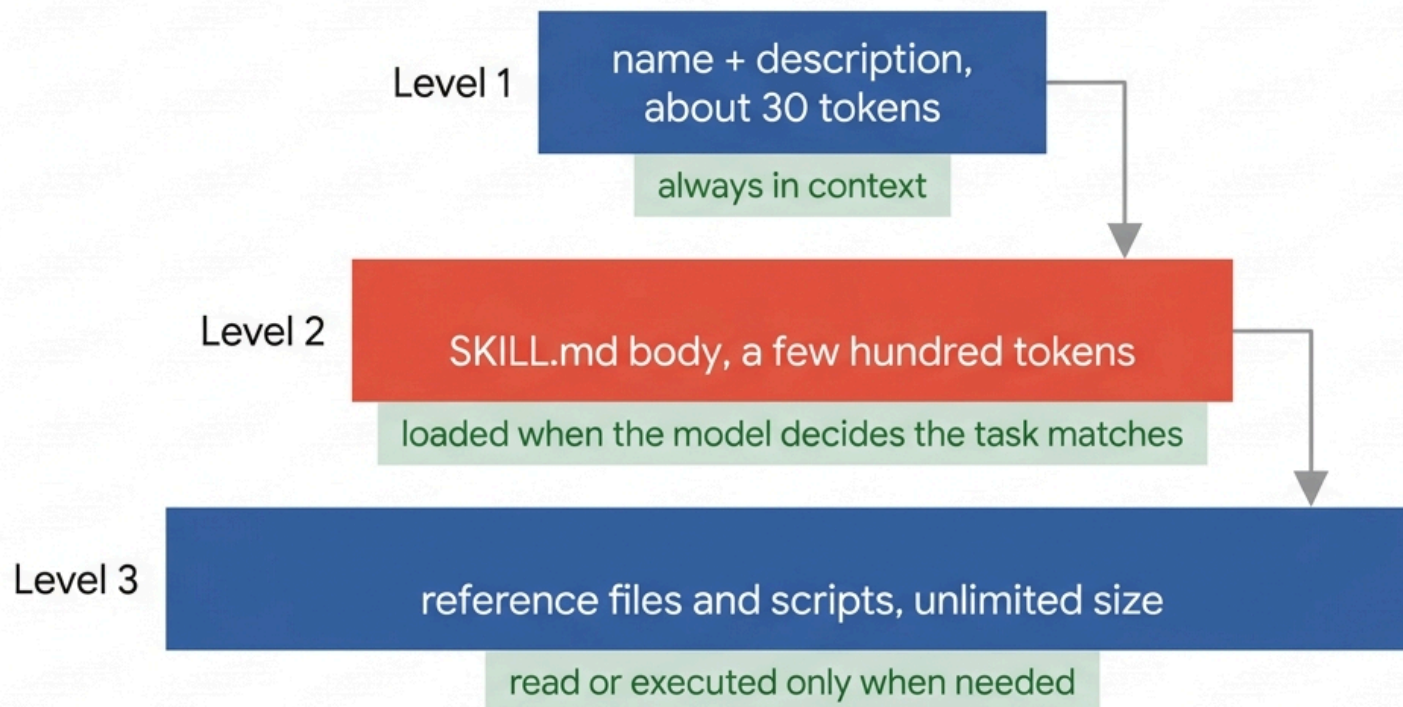
ทางเลือกที่ดี: Skills

โพลเดอร์ที่มีคำสั่ง สคริปต์ และเทมเพลต ซึ่งเอเจนต์จะ **โหลด**
เมื่อต้อง ใช้เท่านั้น

- ไม่กินบริบทเมื่อไม่ได้ใช้
- แก้ไขได้ทันที เป็นแค่ไฟล์ข้อความ
- เก็บใน git ได้ แชร์กันในทีมได้
- พกพาข้าม harness ได้

เชื่อมกลับสัปดาห์ที่ 9: นี่คือการยกย่อง "ถ่ายออกนอกบริบท" ที่เราเรียนไปแล้ว แต่ทำอย่างเป็นระบบและนำกลับมาใช้ซ้ำได้

Progressive disclosure



unlimited depth of expertise at near-zero idle context cost

เปรียบเทียบกับ RAG

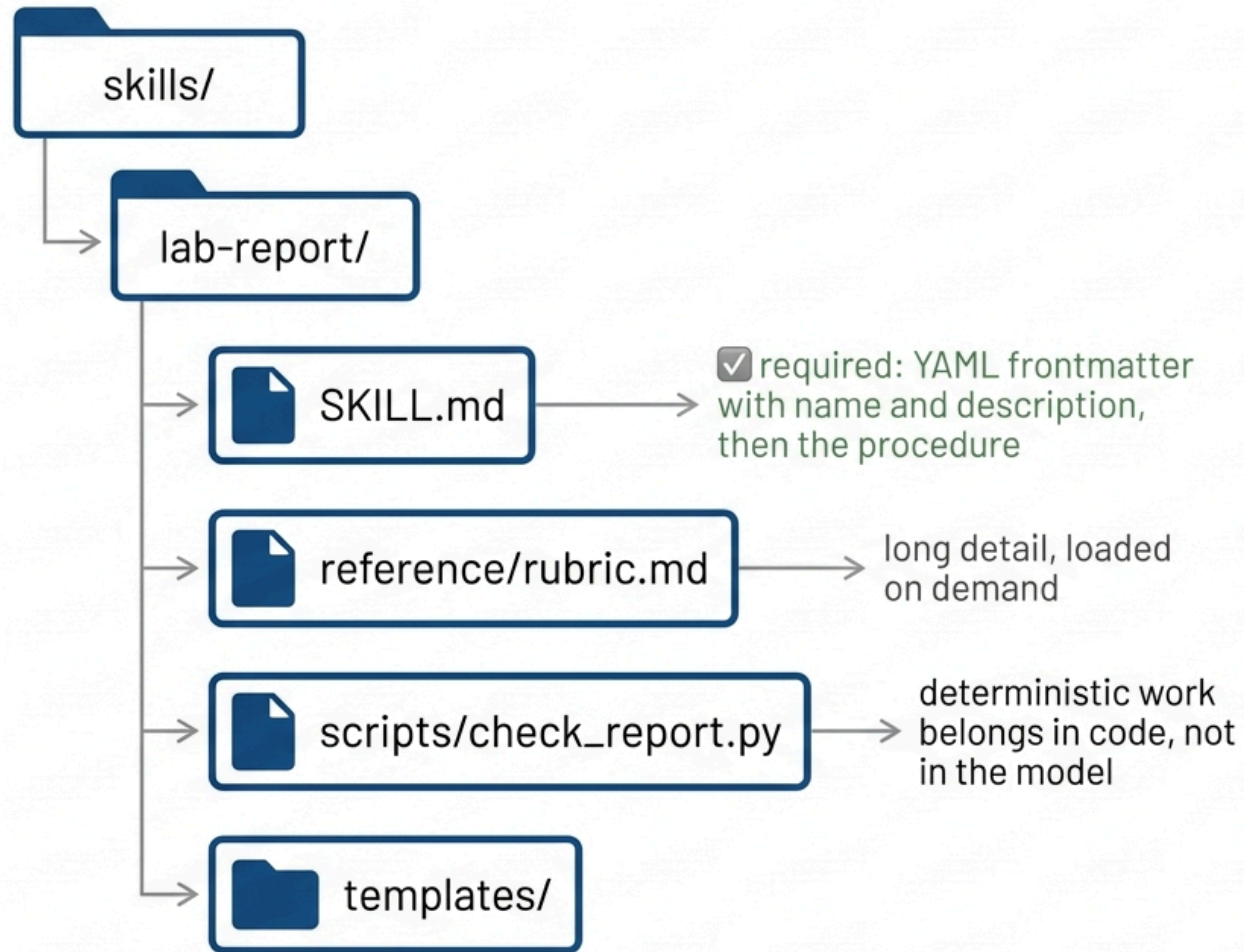
ทั้ง Skills และ RAG คือการเลือกใส่ข้อมูลเข้าบริบทเมื่อจำเป็น แต่วางกันที่ ใครเป็นคนเลือก

	RAG (สัปดาห์ที่ 10)	Skills
วิธีเลือก	ความคล้ายของเวกเตอร์	โมเดลอ่าน description แล้วตัดสินใจ
แม่นยำเมื่อ	คลังใหญ่ ขอบเขตไม่ชัด	ขอบเขตของงานชัดเจน
พังเมื่อ	คำถามใช้คำต่างจากเอกสาร	description เขียนไม่ดี
แก้ไขโดย	ปรับ chunking, hybrid, rerank	แก้ข้อความ description
เนื้อหา	ข้อเท็จจริง	วิธีทำงาน (procedure)

ความต่างที่สำคัญที่สุด: RAG ให้ ข้อเท็จจริง ส่วน Skills ให้ วิธีทำ

"ค่าคงที่ของพลังค์คืออะไร" คืองานของ RAG

"ตั้งค่าไฟล์อินพุต VASP สำหรับการคำนวณ relaxation อย่างไร" คืองานของ Skill



! CRITICAL INSTRUCTION

the description field is the **ONLY** thing the model sees when deciding to use this skill.

Write it as:

what it does + use when ...
+ do not use for ...

โครงสร้างจริงของ Skill

```
---
name: vasp-input
description: สร้างและตรวจไฟล์อินพุตของ VASP (INCAR, KPOINTS, POSCAR)
  ใช้เมื่อผู้ใช้ขอตั้งค่าการคำนวณ DFT หรือถามถึงแท็กใน INCAR
  ห้ามใช้กับการวิเคราะห์ผลลัพธ์ที่คำนวณเสร็จแล้ว
---

# การสร้างไฟล์อินพุต VASP

## ขั้นตอน
1. ตรวจสอบว่ามี POSCAR อยู่แล้วหรือไม่ ถ้าไม่มีให้ถามผู้ใช้ ห้ามสร้างเอง
2. เลือกเทมเพลตจาก `templates/` ตามชนิดการคำนวณ
3. รัน `python scripts/make_incar.py --type relax` เพื่อสร้างไฟล์
4. ตรวจสอบผลด้วย `python scripts/validate.py` ก่อนบอกผู้ใช้ว่าเสร็จ

## ข้อควรระวัง
- ตรวจสอบ ENCUT ให้สูงกว่าค่าที่ POTCAR แนะนำอย่างน้อย 1.3 เท่า
- รายละเอียดแท็กทั้งหมดอยู่ใน `reference/incar-tags.md`
```

สังเกตขั้นที่ 4 เราบังคับให้เอเจนต์ตรวจด้วยโปรแกรมก่อนรายงานผล นี่คือการแก้ปัญหา "มั่นใจแต่ผิด" จากสัปดาห์ที่แล้ว โดยเขียนไว้ใน skill

เขียน description ให้ดี คือ 80% ของงาน

description คือสิ่งเดียวที่โมเดลเห็นตอนตัดสินใจว่าจะเรียก skill นี้หรือไม่

แย่

description: ช่วยเรื่อง VASP

กว้างเกินไป โมเดลไม่รู้ว่าจะเมื่อไรควรใช้

ดี

description: สร้างและตรวจไฟล์อินพุต
ของ VASP (INCAR, KPOINTS, POSCAR)
ใช้เมื่อผู้ใช้ขอตั้งค่าการคำนวณ DFT
หรือถามถึงแท็กใน INCAR
ห้ามใช้กับการวิเคราะห์ผลลัพธ์

สูตรที่ใช้ได้ผล: [ทำอะไร] + [ใช้เมื่อ ...] + [ห้ามใช้กับ ...] + [คำที่ผู้ใช้มักพูด]

ข้อผิดพลาดที่พบบ่อยที่สุด: เขียนเนื้อหาใน SKILL.md ดีมาก แต่เขียน description กว้าง ๆ ผลคือ skill ไม่เคยถูกเรียกใช้เลย และไม่มีข้อความผิดพลาดใด ๆ ให้เห็น คุณจะคิดว่ามันทำงานอยู่ ทั้งที่มันไม่เคยถูกโหลดเลยแม้แต่ครั้งเดียว

แบบฝึกหัด 2.1: วิจัย description

แต่ละ description ต่อไปนี้มีปัญหาอะไร และจะเกิดอะไรขึ้นในทางปฏิบัติ

```
# ก
description: เครื่องมือช่วยงานวิจัย

# ข
description: ใช้ skill นี้เสมอทุกครั้งที่ใช้ถามอะไรก็ตาม

# ค
description: วิเคราะห์ข้อมูล XRD โดยใช้วิธี Rietveld refinement
ผ่านโปรแกรม GSAS-II เวอร์ชัน 5.2 ซึ่งต้องติดตั้ง Python 3.10
และไลบรารี numpy scipy matplotlib h5py pillow ...

# ง
description: จัดรูปแบบโค้ด Python ตามมาตรฐานของแล็บ
ใช้เมื่อผู้ใช้ขอจัดรูปแบบโค้ดหรือขอให้ทำให้โค้ดอ่านง่ายขึ้น
```

เฉลย 2.1

#	ปัญหา	ผลในทางปฏิบัติ
ก	กว้างจนไร้ความหมาย	โมเดลไม่มีสัญญาณให้ตัดสินใจ จะเรียกมั่วบ้าง ไม่เรียกบ้าง ผลลัพธ์ไม่คงเส้นคงวา
ข	บังคับให้เรียกทุกครั้ง	ทำลายจุดประสงค์ของ progressive disclosure ทั้งหมด เนื้อหาระดับ 2 จะถูกโหลดตลอดเวลา เท่ากับยัดใส่พรมปูระบบ
ค	ยัดรายละเอียดระดับ 3 มาไว้ระดับ 1	กินบริบทตลอดเวลาโดยไม่จำเป็น รายการโลบริควรรออยู่ใน reference/ หรือใน SKILL.md
ง	description ดี แต่เลือกกลไกผิด	การจัดรูปแบบโค้ดควรเป็น hook ที่รันหลังแก้ไฟล์ทุกครั้ง ไม่ใช่ skill ที่รอให้โมเดลนึกขึ้นได้

ข้อ ง คือบทเรียนที่ลึกที่สุด คำถามไม่ใช่แค่ "เขียน description ยังไงให้ดี" แต่คือ "นี่ควรเป็น skill หรือเปล่า" งานที่ ต้องเกิดขึ้นทุกครั้ง ไม่ควรฝากไว้กับดุลพินิจของ โมเดล เราจะคุยเรื่องการเลือกกลไกอย่างเป็นระบบในชั่วโมงหน้า

แนวปฏิบัติในการเขียน Skill

ควรทำ

- ให้ SKILL.md สั้น ย้ายรายละเอียดไปไฟล์อ้างอิง
- เขียนเป็นขั้นตอนที่ทำได้ ไม่ใช่เรียงความ
- ใส่สคริปต์สำหรับงานที่ทำซ้ำและต้องแม่นยำ
- ระบุกรณีขอบและข้อควรระวัง
- บังคับให้ตรวจสอบผลด้วยโปรแกรมก่อนรายงาน
- ทดสอบด้วยงานจริง 5 ถึง 10 งาน

ไม่ควรทำ

- ยัดคู่มือ 50 หน้าลง SKILL.md
- อธิบายสิ่งที่โมเดลรู้อยู่แล้ว เช่น ไวยากรณ์ Python
- ให้ skill ทำหลายเรื่องที่ไม่เกี่ยวกัน
- ลืมทดสอบว่ามันถูกเรียกใช้จริงหรือไม่

เมื่อไรควรใช้ Skill เทียบกับ MCP:

Skill = ความรู้และวิธีการทำงาน ที่เอเจนต์ทำได้ด้วยเครื่องมือที่มีอยู่แล้ว (ไฟล์ข้อความ)

MCP = การเชื่อมต่อกับระบบภายนอกที่ต้องมีโค้ดด้วย (เซิร์ฟเวอร์ที่รันอยู่)

งานจริงมักใช้ทั้งคู่: **MCP** ให้ความสามารถ ส่วน **Skill** สอนว่าจะใช้ความสามารถนั้นอย่างไร

สรุปชั่วโมงที่ 2

- **Skills** คือความรู้เฉพาะทางในรูปแบบโพสเตอร์ ที่โหลดแบบฉบับอัตโนมัติเมื่อจำเป็น
- 3 ระดับ: ชื่อและคำอธิบาย (~30 โทเคน) → เนื้อหา SKILL.md → ไฟล์อ้างอิงและสคริปต์
- **RAG** ให้ข้อเท็จจริง Skills ให้วิธีทำ
- **description ที่เขียนดี** คือสิ่งที่กำหนดว่า skill จะถูกใช้จริงหรือไม่ สูตร: ทำอะไร + ใช้เมื่อ + ห้ามใช้กับ + คำที่ผู้ใช้พูด
- ความล้มเหลวของ skill **ไม่มีข้อความผิดพลาด** จึงต้องวัดอัตราการเรียกใช้
- งานที่ต้องเกิดขึ้นทุกครั้ง **ไม่ควรเป็น skill**

พัก 10 นาที

ชั่วโมงที่ 3

Plugins และการเลือกกลไก

Plugin: รวมทุกอย่างเป็นชุดเดียว

```
my-lab-plugin/  
  .claude-plugin/  
    plugin.json      <- ข้อมูลแพ็คเกจ ชื่อ เวอร์ชัน คำอธิบาย  
skills/  
  vasp-input/SKILL.md <- ความรู้เฉพาะทาง  
  paper-review/SKILL.md  
commands/  
  submit-job.md      <- คำสั่งลัด /submit-job  
agents/  
  reviewer.md        <- นิยามเอเจนต์ย่อยเฉพาะทาง  
hooks/  
  hooks.json         <- กฎอัตโนมัติ  
.mcp.json            <- เซิร์ฟเวอร์ MCP ที่มากับชุดนี้
```

ประโยชน์กับทีมและห้องเรียน

- นักศึกษาใหม่ติดตั้งครั้งเดียว ได้สภาพแวดล้อมเหมือนกันทั้งชั้น
- อัปเดตผ่าน git ทุกคนได้เวอร์ชันเดียวกัน
- ความรู้ของแล็บถูกเก็บเป็นโค้ด ไม่ใช่ในหัวคนใดคนหนึ่ง

ตัวอย่าง: คำสั่งลัดและ hook

commands/submit-job.md

```
---  
description: ส่งงานคำนวณเข้าคิว SLURM  
---
```

ตรวจไฟล์อินพุตในไดเรกทอรีปัจจุบัน
สร้างสคริปต์ SLURM จากเทมเพลต
แสดงให้ผู้ใช้นั่นก่อน แล้วจึง sbatch
รายงานหมายเลขงานที่ได้

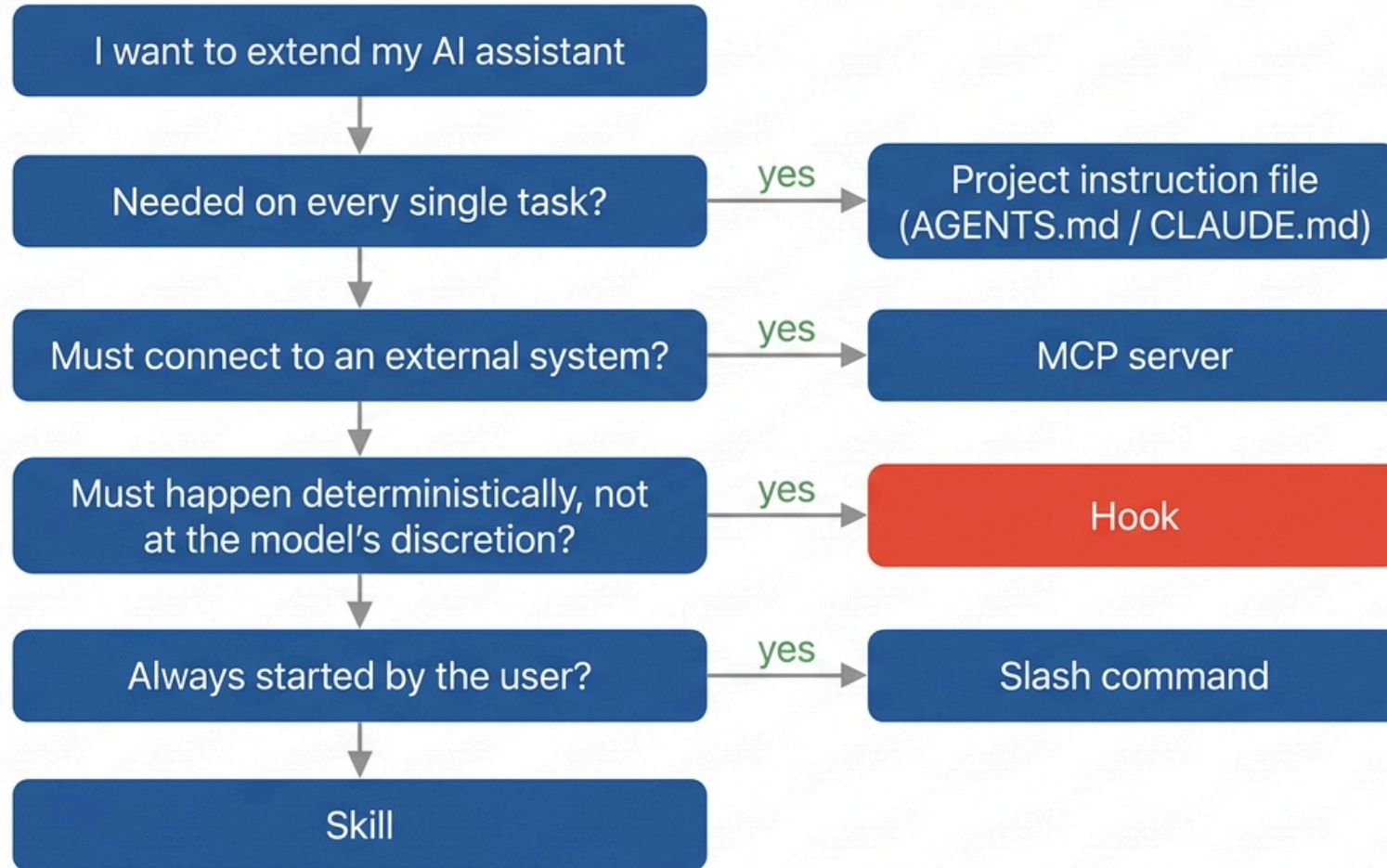
เรียกใช้ด้วย /submit-job

hooks/hooks.json

```
{  
  "PreToolUse": [{  
    "matcher": "Bash",  
    "hooks": [{  
      "type": "command",  
      "command": "python3 block_secrets.py"  
    }]  
  }]  
}
```

บล็อกคำสั่งที่จะทำให้ API key หลุด **ทุกครั้ง** โดยไม่ต้องหวังพึ่ง
พรมปิด

Which extension mechanism?



เปรียบเทียบกลไกขยายความสามารถ

กลไก	เป็นอะไร	โหลดเมื่อไร	เหมาะกับ
ไฟล์คำสั่งโปรเจกต์	ข้อความ	ทุกเซสชัน	กติกาที่ใช้กับทุกงานในโปรเจกต์
Skill	ไฟล์เดอร์ข้อความ + สคริปต์	เมื่อโมเดลเห็นว่าตรงงาน	ความรู้เฉพาะทางที่ใช้เป็นบางครั้ง
คำสั่งลัด	ข้อความ	เมื่อผู้ใช้พิมพ์เรียก	ขั้นตอนที่ผู้ใช้เริ่มเอง
เอเจนต์ย่อย	นิยามเอเจนต์	เมื่อถูกมอบหมายงาน	งานสำรวจที่กินทรัพยากรมาก
MCP server	โปรแกรมที่รันอยู่	เชื่อมต่อตอนเริ่ม	ต่อกับระบบภายนอก
Hook	โปรแกรม	ณ จุดที่กำหนด อัตโนมัติ	กฎที่ต้องบังคับใช้แน่นอน

คำถามคัดเลือก 4 ข้อ ตามลำดับ

1. ต้องใช้ทุกครั้งไหม → ไฟล์คำสั่งโปรเจกต์
2. ต้องเชื่อมระบบภายนอกไหม → MCP
3. ต้องเกิดขึ้นแน่นอนโดยไม่พึ่งดุลพินิจโมเดลไหม → Hook
4. ผู้ใช้เป็นคนเริ่มเองเสมอไหม → คำสั่งลัด

นอกนั้นคือ Skill

แบบฝึกหัด 3.1: เลือกกลไกให้ถูก

#

สถานการณ์

- ก ทุกไฟล์ Python ที่เอเจนต์แก้ ต้องถูกจัดรูปแบบด้วย black เสมอ
- ข ทีมใช้ pytest ไม่ใช่ unittest และห้ามใช้ print ในโค้ดจริง
- ค ต้องดึงข้อมูลผลการทดลองจากฐานข้อมูลของแล็บที่รันอยู่บนเซิร์ฟเวอร์
- ง ขั้นตอนเตรียมไฟล์อินพุต VASP ซึ่งใช้เดือนละไม่กี่ครั้ง แต่ซับซ้อน
- จ นักศึกษาอยากสั่ง "ตรวจรายงานผมก่อนส่ง" ด้วยคำสั่งสั้น ๆ
- ฉ ต้องอ่านโค้ด 200 ไฟล์เพื่อตอบคำถามเดียว โดยไม่ให้บริบทหลัก

เฉลย 3.1

#	คำตอบ	เหตุผล
ก	Hook (หลังใช้เครื่องมือ)	ต้องเกิด ทุกครั้ง ไม่ฟังดุลพินิจโมเดล คำถามข้อ 3
ข	ไฟล์คำสั่งโปรเจกต์ (AGENTS.md)	เป็นกติกาที่ใช้กับ ทุกงาน ในโปรเจกต์นี้ คำถามข้อ 1
ค	MCP server	ต้องเชื่อมระบบภายนอกที่ต้องมีโค้ดด้วย คำถามข้อ 2
ง	Skill	ความรู้เฉพาะทาง ใช้เป็นบางครั้ง ชับซ้อนพอที่จะมี reference และ script
จ	คำสั่งลัด	ผู้ใช้เป็นคนเริ่มเองเสมอ คำถามข้อ 4
ฉ	เอเจนต์ย่อย	งานสำรวจที่กินบริบทมาก คืบแค่ข้อสรุป

ก เทียบ ง คือคู่ที่คนสลับกันบ่อยที่สุด ทั้งคู่เกี่ยวกับ "วิธีทำงานเฉพาะของทีม" แต่ต่างกันที่ **ความแน่นอน**ที่ต้องการการจัดรูปแบบโค้ดต้องเกิดขึ้น 100% ของครั้ง → hook
การเตรียมไฟล์ VASP เกิดเฉพาะเมื่อผู้ใช้งานนั้น → skill

ช่วงลงมือทำ: labs/w13_skills_plugin/

ปลั๊กอินตัวอย่างที่ครบทั้ง 4 กลไก รันได้ทันทีโดยไม่ต้องติดตั้งอะไร

```
# 1. ตรรกะของ skill ทำงานถูกต้องไหม
python plugin/skills/lab-report/scripts/check_report.py

# 2. hook บล็อกคำสั่งอันตรายได้จริงไหม
python plugin/hooks/block_secrets.py --self-check
echo '{"tool_input":{"command":"cat .env"}}' | python plugin/hooks/block_secrets.py

# 3. description ทำให้ skill ถูกเรียกถูกจังหวะไหม
python eval_skill.py --self-check
```

`eval_skill.py` คือหัวใจของแล็บนี้ มันทำให้ปัญหา "skill ไม่เคยถูกเรียกและไม่มี error ให้เห็น" กลายเป็นตัวเลขที่วัดได้
รันกับโมเดลจริง ดูว่าผิดพลาด false negative (description แคบไป) หรือ false positive (กว้างไป) แล้วแก้ SKILL.md แล้ววัดซ้ำ
นี่คืออุปกรณ์ evaluator-optimizer ของสัปดาห์หน้า ที่เราใช้กับตัวเอง

สรุป

- **Harness** คือโปรแกรมรอบโมเดล และกำหนดความสามารถจริงพอ ๆ กับตัวโมเดล
- องค์ประกอบ: พรอมป์ระบบ, เครื่องมือ, ชั้นลิทธี, ตัวจัดการบริบท, เอเจนต์ย่อย, hooks
- **Skills** คือความรู้เฉพาะทางที่โหลดแบบฉบับได้ จึงไม่กินบริบทเมื่อไม่ได้ใช้
- **description** คือสิ่งเดียวที่โมเดลเห็นตอนตัดสินใจ และความล้มเหลวไม่มีข้อความเตือน
- **Plugins** รวม skills, คำสั่ง, เอเจนต์ย่อย, MCP และ hooks เป็นชุดที่แจกจ่ายได้
- เลือกกลไกด้วยคำถาม 4 ข้อ: ทุกครั้งไหม / ระบบภายนอกไหม / ต้องแน่นอนไหม / ผู้ใช้เริ่มไหม
- อะไรที่ต้องเกิดขึ้นแน่นอน ให้ใช้ hook ไม่ใช่พรอมป์

สัปดาห์หน้า: เราจะเอาทุกอย่างมาประกอบเป็นเวิร์กโฟลว์ที่ทำงานเองโดยไม่มีคนนั่งเฝ้า พร้อมคุยเรื่องความปลอดภัยและจริยธรรมที่ตามมา

การบ้านสัปดาห์ที่ 13

ส่วนที่ 1: ออกแบบ

1. ออกแบบนโยบายสิทธิแบบฝึกหัด 1.1 สำหรับงานของคุณเอง อย่างน้อย 8 คำสั่ง
2. เลือกกลไกให้ 5 สถานการณ์ในสาขาของคุณ พร้อมอธิบายด้วยคำถาม 4 ข้อ

ส่วนที่ 2: แล็บ labs/w13_skills_plugin/

1. เขียน Skill ของคุณเอง 1 ชุด ต้องมี SKILL.md + ไฟล์อ้างอิง 1 ไฟล์
 - สคริปต์ที่รันได้ 1 ตัวพร้อม self-check
1. เขียนชุดทดสอบ 10 คำขอใน eval_skill.py (ควรเรียก 5 ไม่ควรเรียก 5)
2. รันวัด แก่ description แล้ววัดซ้ำ ส่งตารางคะแนนก่อนและหลัง พร้อมอธิบายว่าแก้อะไร
3. รวมเป็น plugin ที่มีคำสั่งลัด 1 คำสั่งและ hook 1 ตัว
4. รัน skill เดียวกันใน harness 2 ตัวที่ต่างกัน บันทึกว่าอะไรพกพาได้และอะไรไม่ได้

จบสัปดาห์ที่ 13